

Utilizing CSP in Nucleo-F722ZE

1. Creating and Configuring a Workspace in STM32CubeIDE: Start by creating a new workspace in STM32CubeIDE and configuring it according to your project requirements. This includes setting up the project structure, selecting the target microcontroller (in this case, Nucleo-F722ZE), and configuring any necessary build settings.

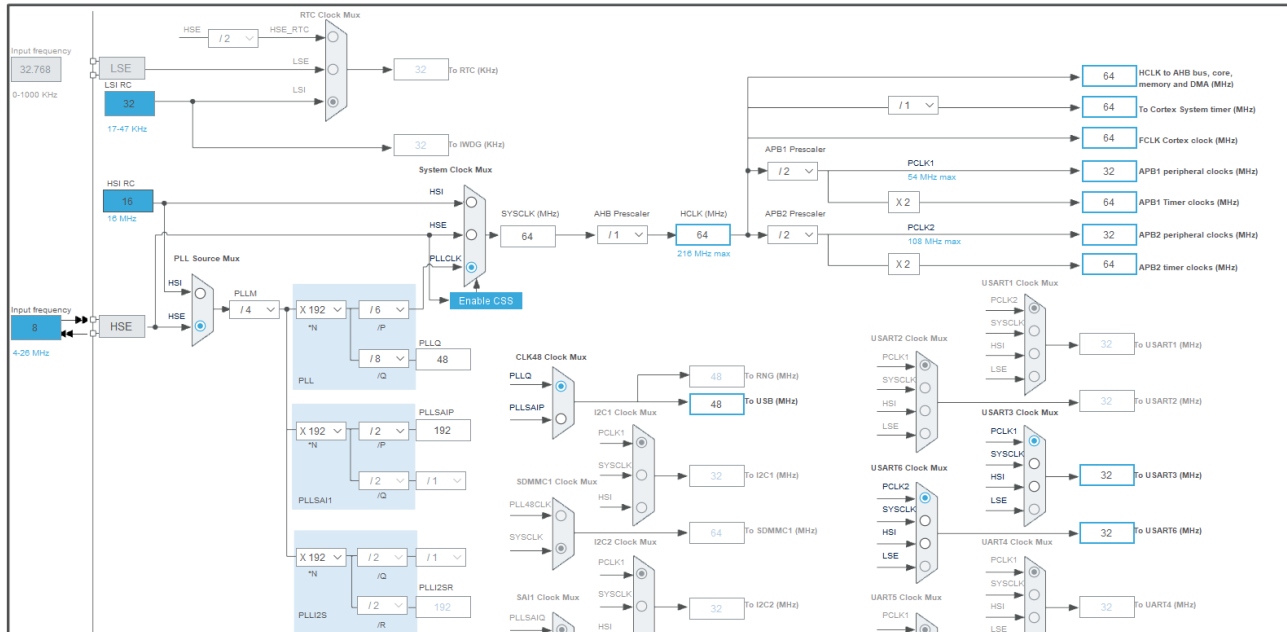


Figure 1 Creating and Configuring

Please check it system Core is correct.

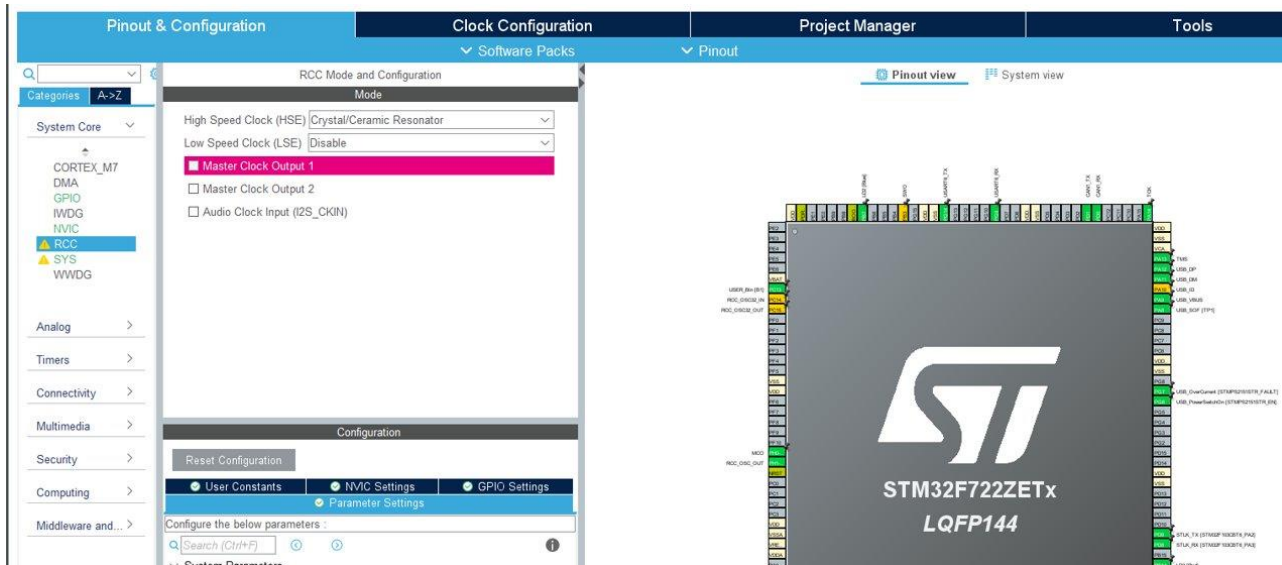


Figure 2 system Core

1.1 After creating your project, navigate to the IOC page to enable CAN. Typically, the Baud Rate used is 1000000 bits/s.

In the case where you use the CAN driver

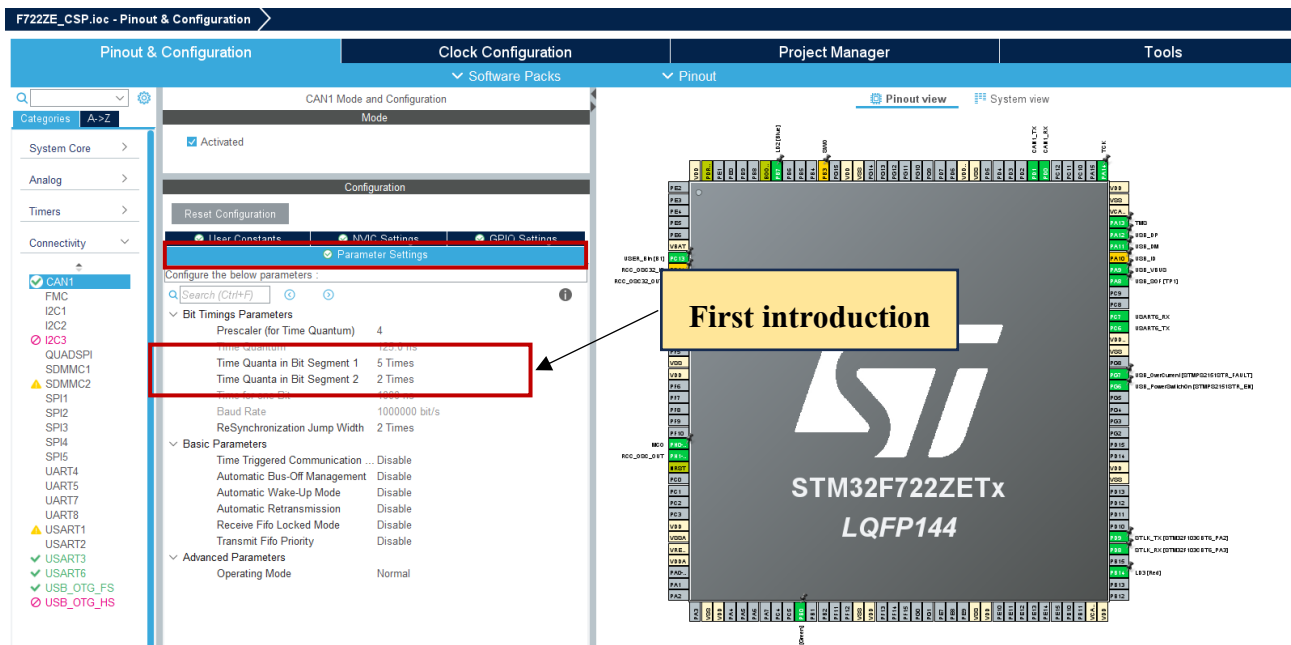


Figure 3 enable CAN

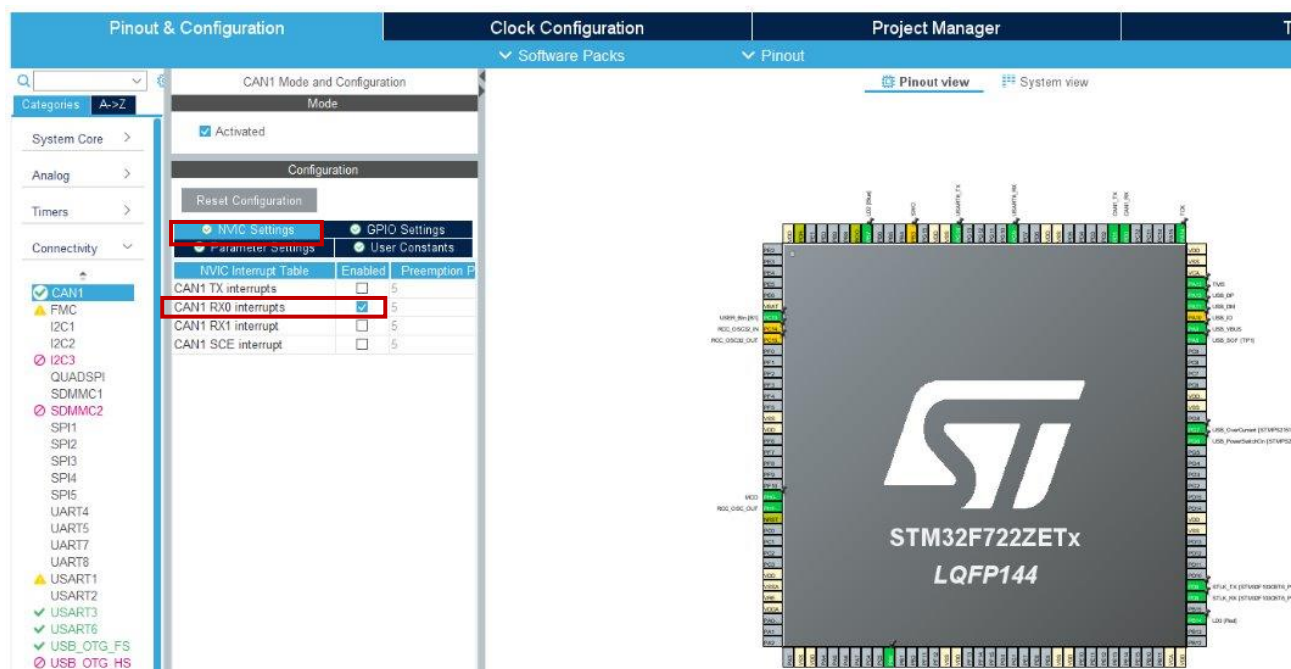


Figure 4 Enable interrupt

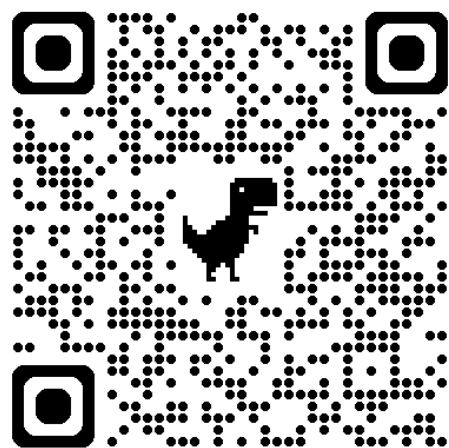


Figure 5 Calculating various

FDCAN Clock

[MHz]

FDCAN Clock Divider

FDCAN_CLOCK_DIV1

▼

Baudrate tolerance

[%]

Nominal rate (CLASSIC CAN)

[kbit/s]

Sampling point (CLASSIC CAN)

[%]

Data rate (FD CAN)

[kbit/s]

Sampling point (FD CAN)

[%]

SOLVE

Table with **NOMINAL (CLASSIC)** bit time:

Baudrate	Prescaler	Time quanta	Time Seg1	Time Seg2	Sample point [%]	Sample point diff [%]
1000	1	80	59	20	75	0
1000	2	40	29	10	75	0
1000	4	20	14	5	75	0

Figure 6 <https://phrynishak.github.io/stm32g-fdcan>

1.2 Enable FREERTOS and select the interface as CMSIS_V1. Navigate to the config parameters and go to Memory Allocation, set it to Dynamic/Static. Enable USE_TRACE_FACILITY and USE_STATS_FORMATTING_FUNCTIONS

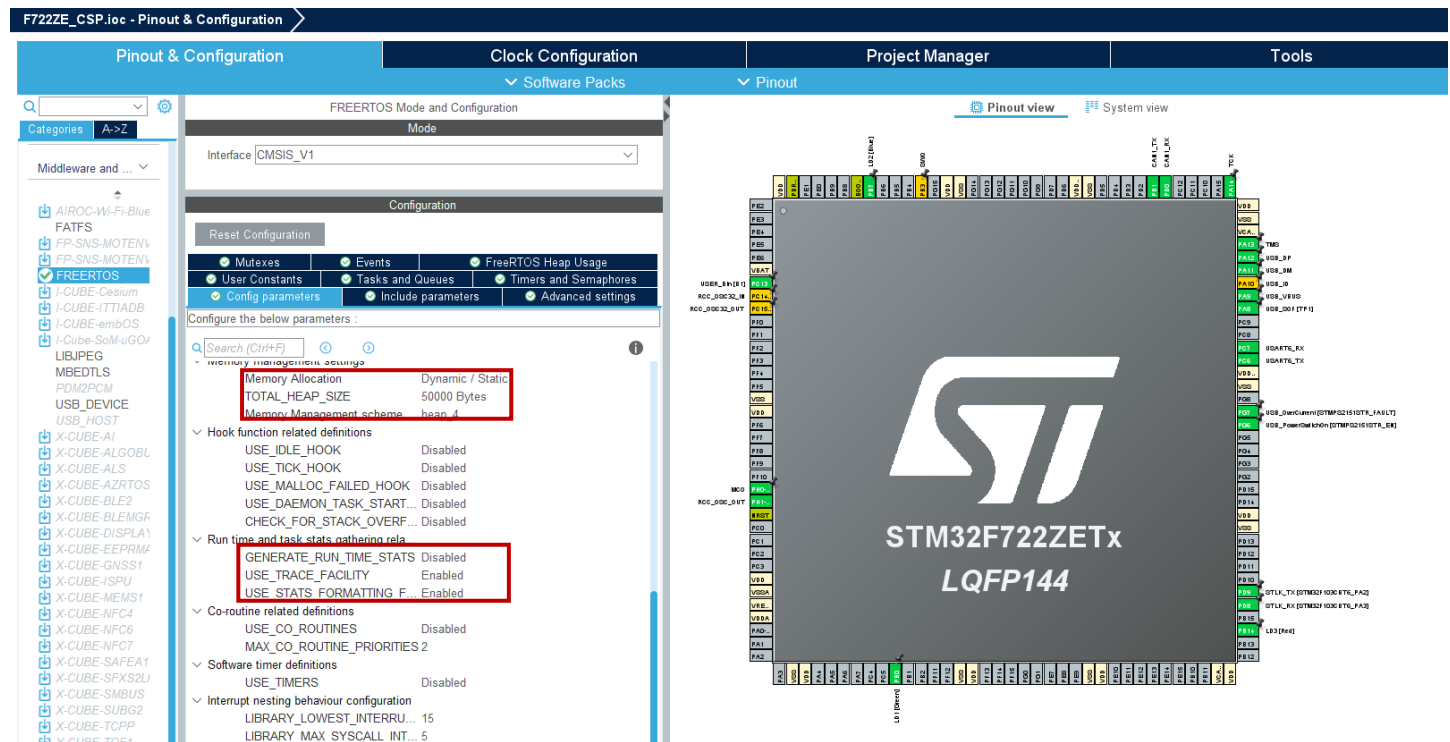


Figure 7 Enable FREERTOS

Warning: The value chosen by the user should take into account the application consumption of RAM otherwise it will lead to link issues! For information: $\text{min value} = \text{configMINIMAL_STACK_SIZE} * 4$

1.3 enable Use Float.

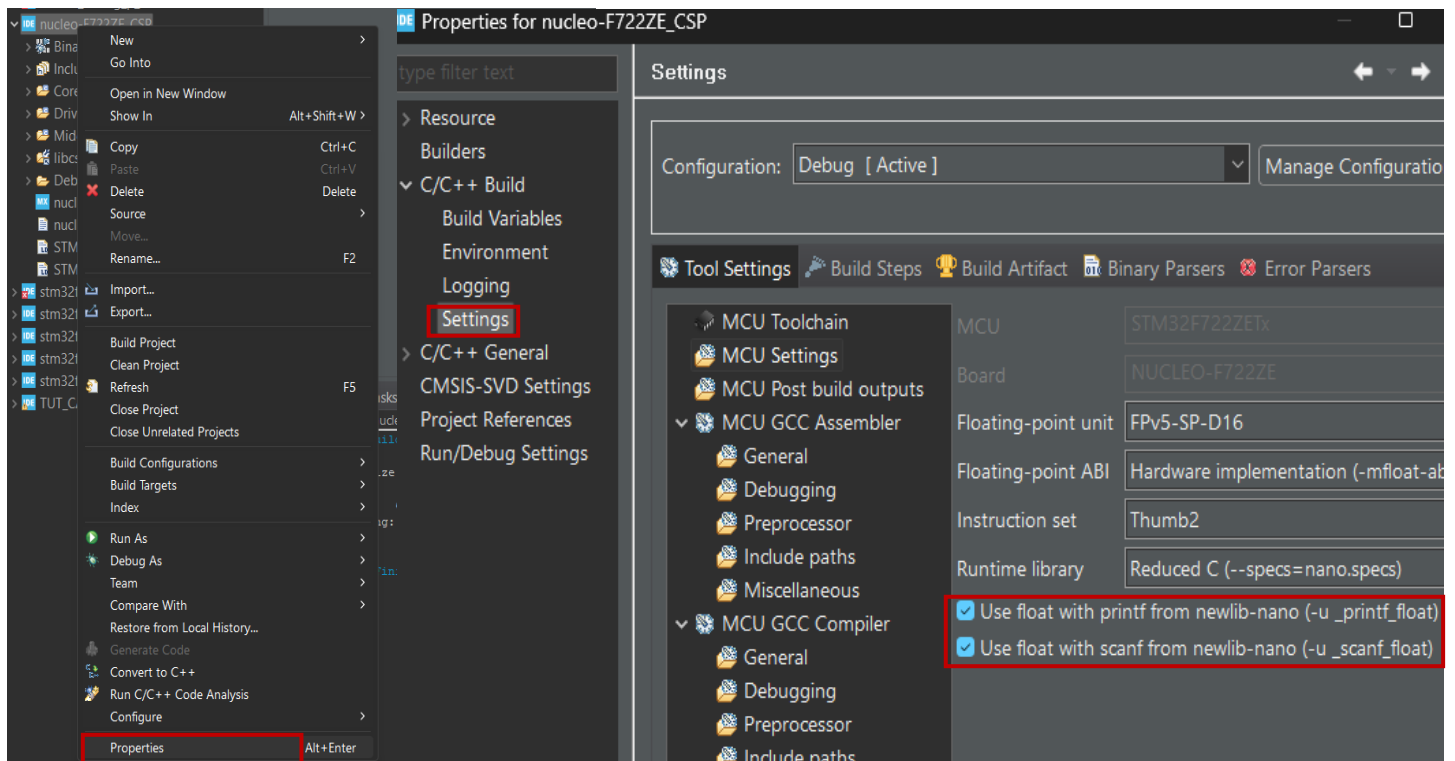


Figure 8 enable Float

2. Importing CSP Library: Once the workspace is set up, import the CSP (Communications Service Provider) library into your project. This involves adding the necessary files and configurations to your project within STM32CubeIDE.

- The next step is to include CSP_1.6 , You can clone CSP_1.6 from the following

Command: git clone https://git.narit.or.th/chaturong/CSP_1.6.git (I've changed the project name)

- Include CSP_1.6 in your project.

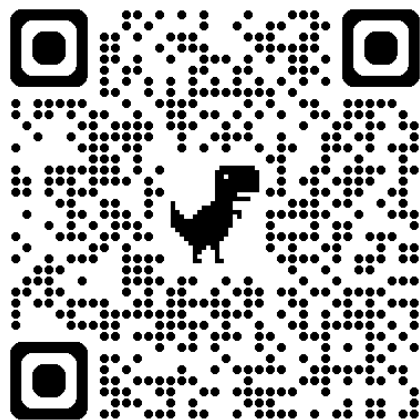


Figure 9 https://git.narit.or.th/chaturong/CSP_1.6.git

include CSP_1.6

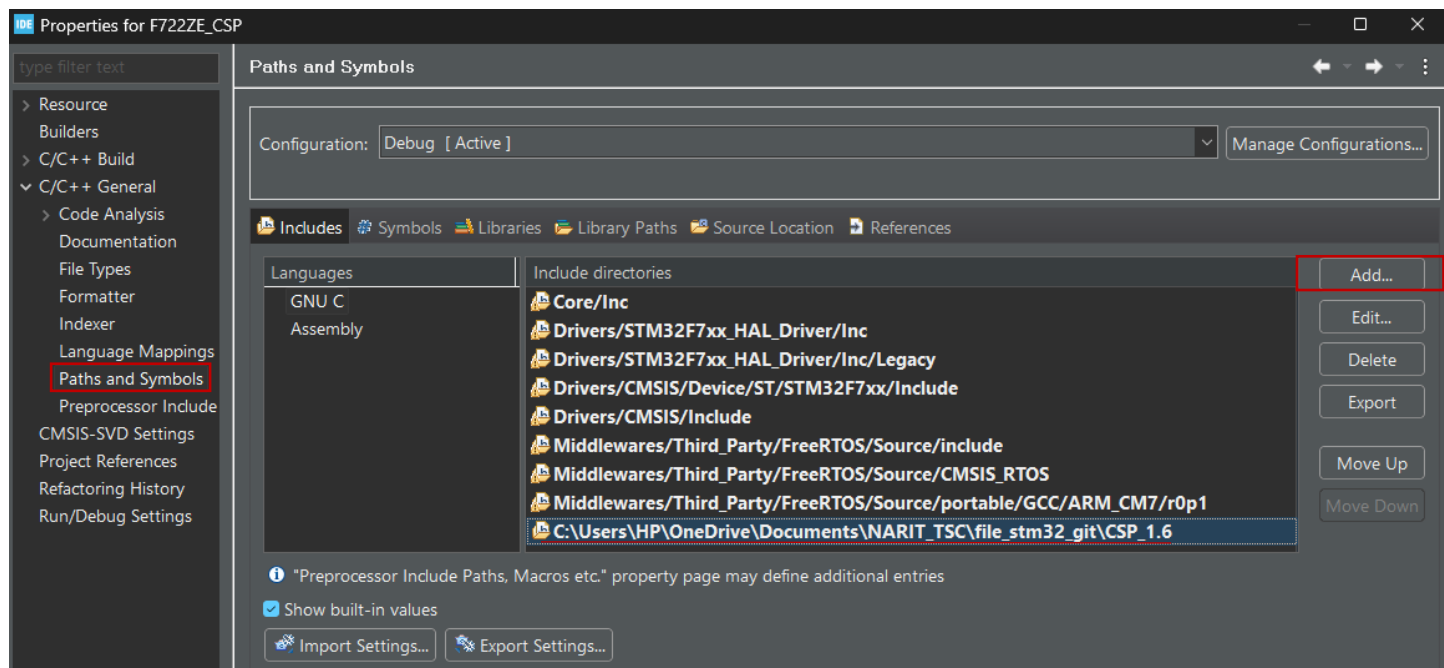


Figure 10 include CSP_1.6

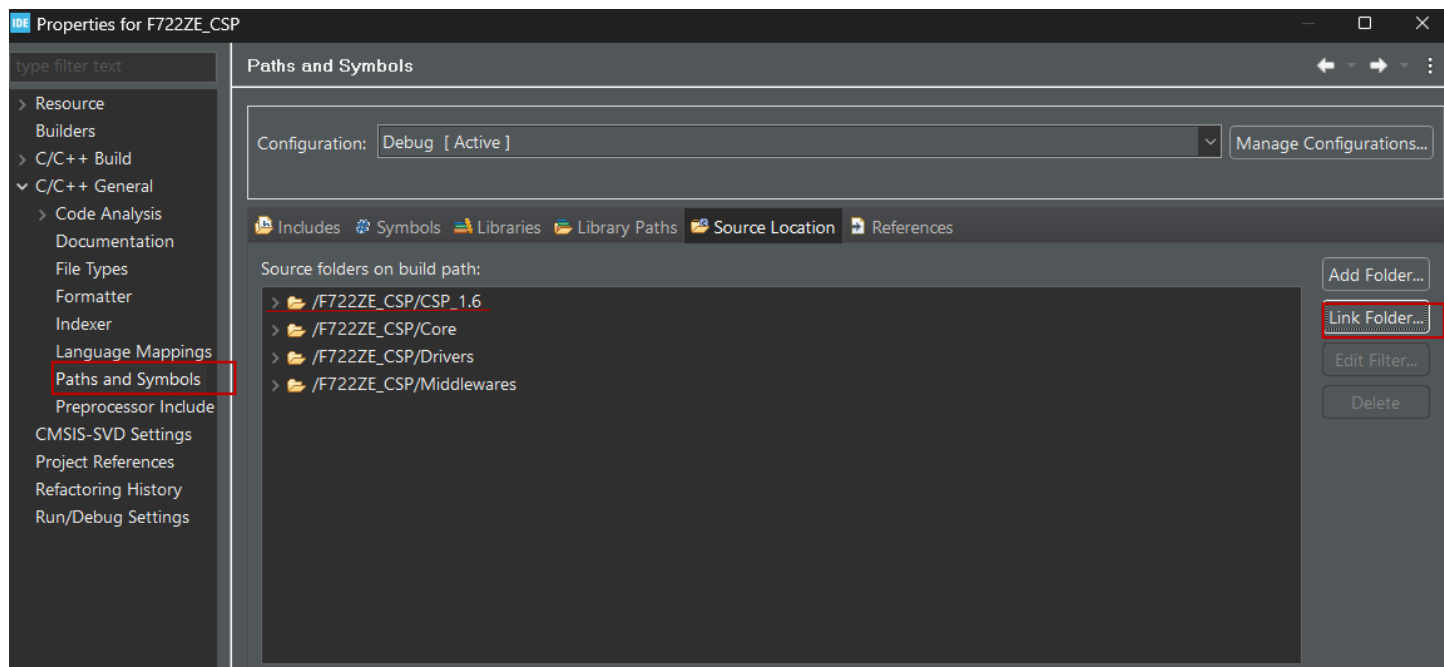


Figure 11 Link Folder CSP_1.6

Exclude Examples

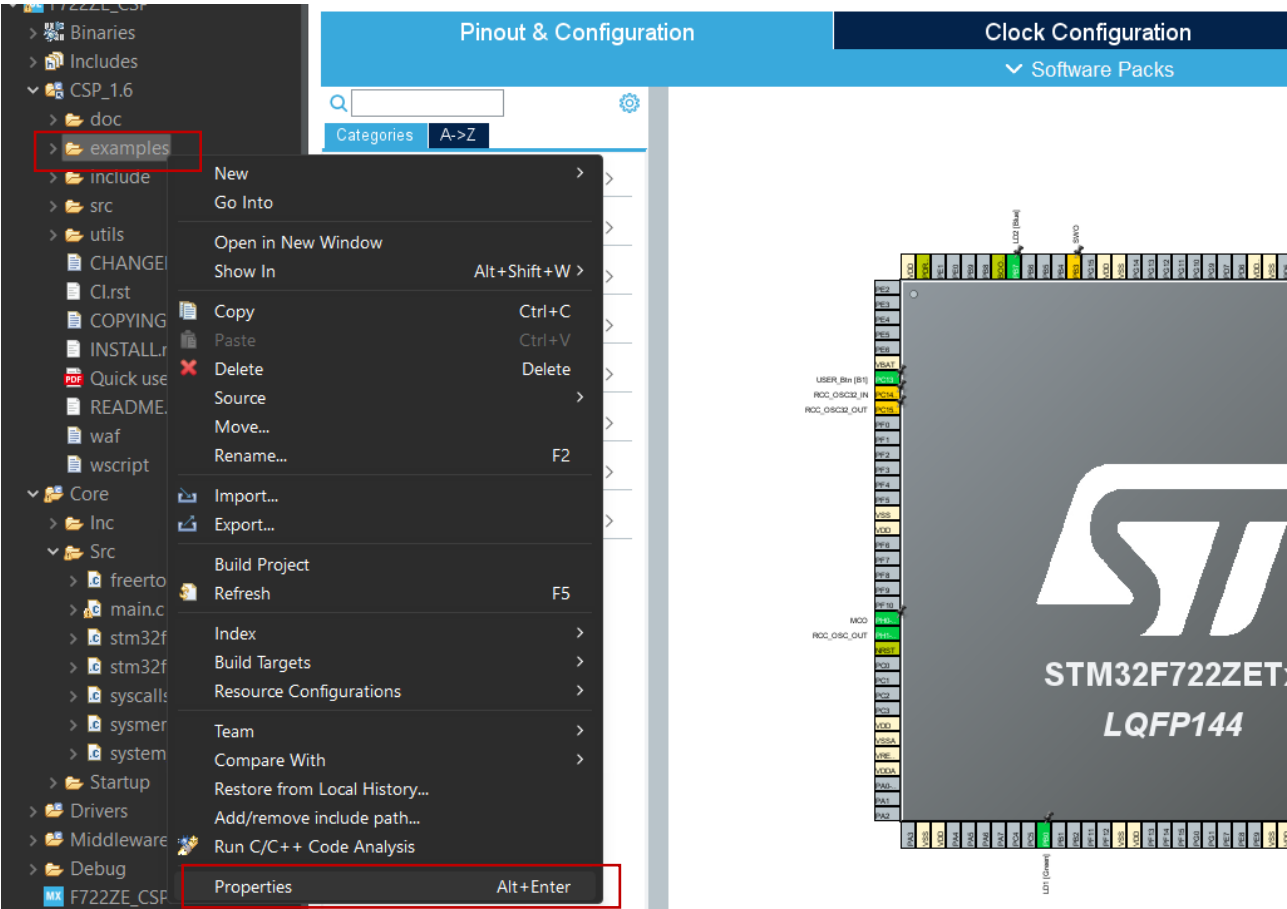


Figure 12 Exclude Examples1

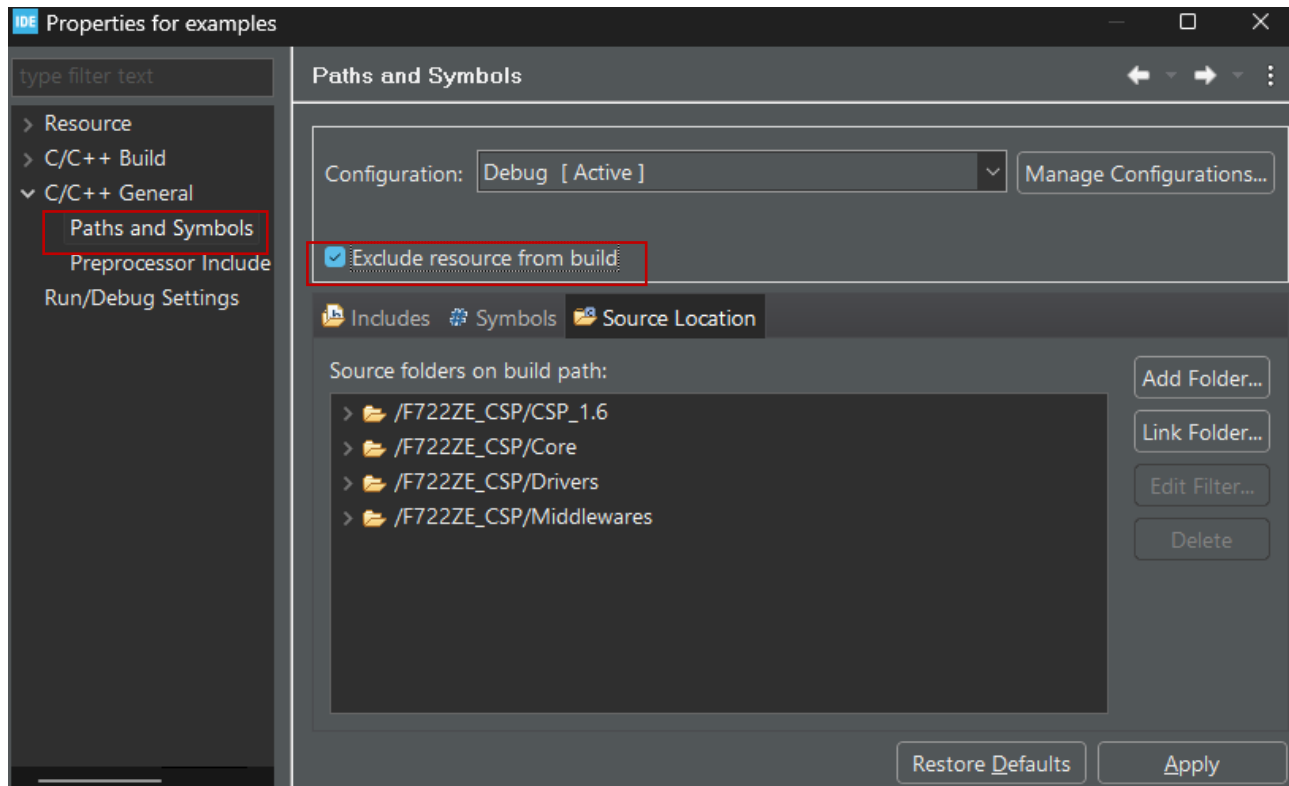


Figure 13 Exclude Examples2

3. Integrating CSP with Nucleo-F722ZE: After importing the CSP library, integrate it with your Nucleo-F722ZE project. This typically involves linking the library, initializing it within your code, and utilizing its functionalities as required for your application. Make sure to configure CSP settings appropriately to meet the communication requirements of your project.

3.1 Including libcsp in main.c for usage.

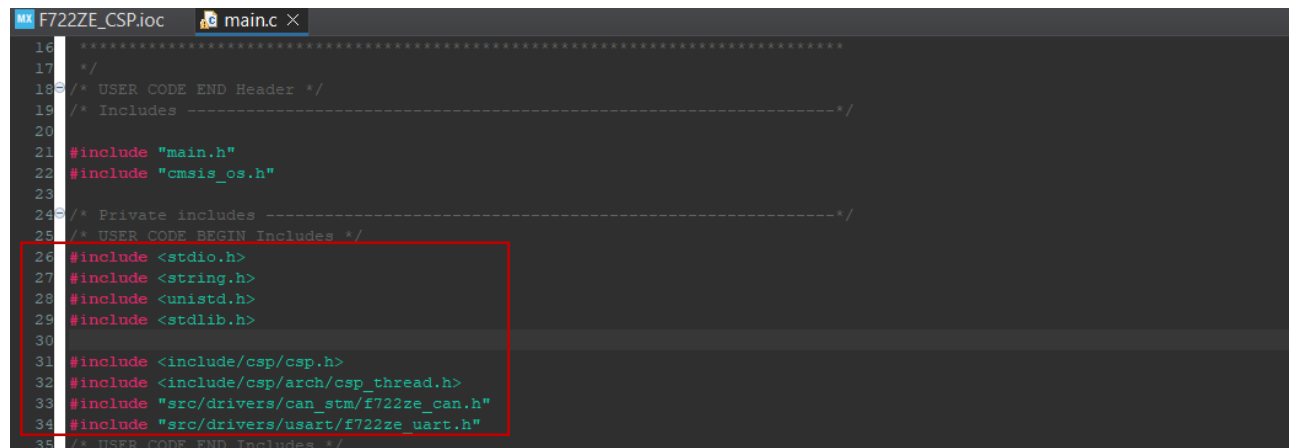


Figure 14 Including libcsp in main.c

3.2 Select the Nucleo model in the csp_autoconfig.h file.

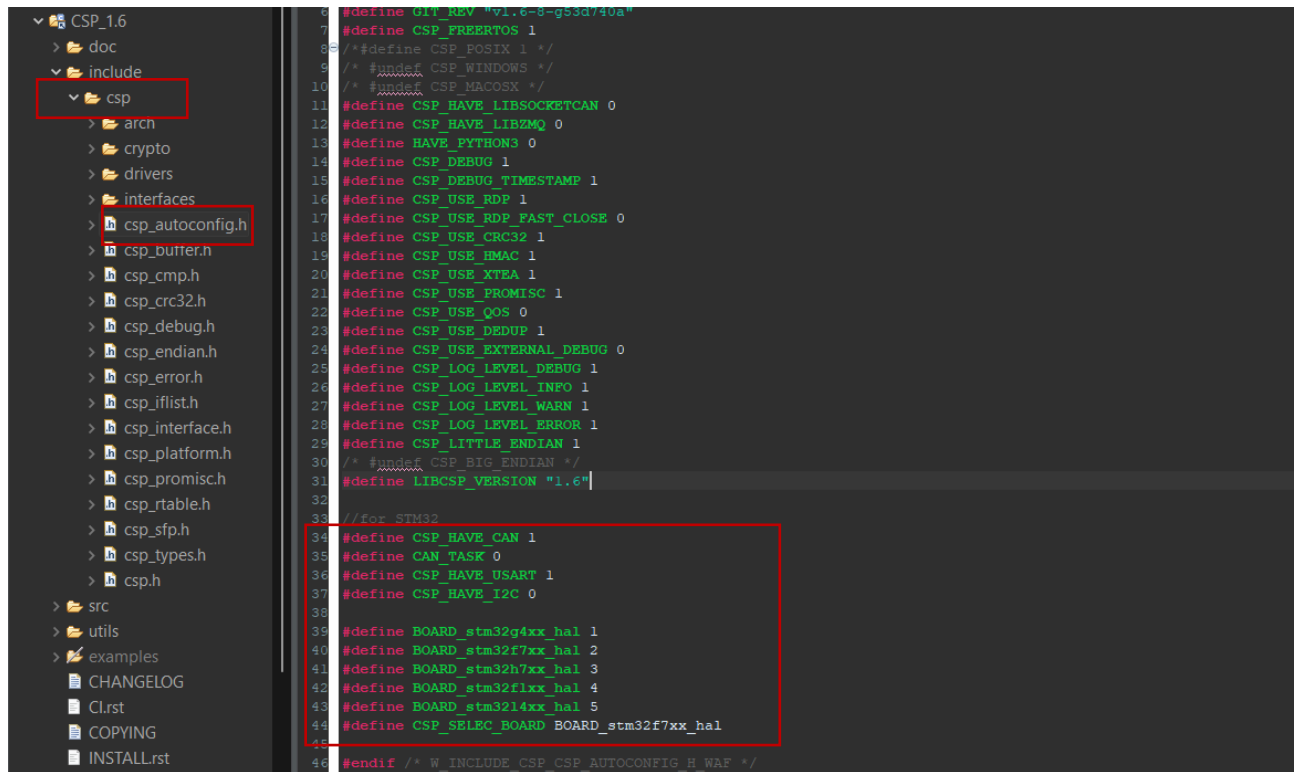


Figure 15 Select the Nucleo model

3.3 Define variables and create functions for testing in the file main.c

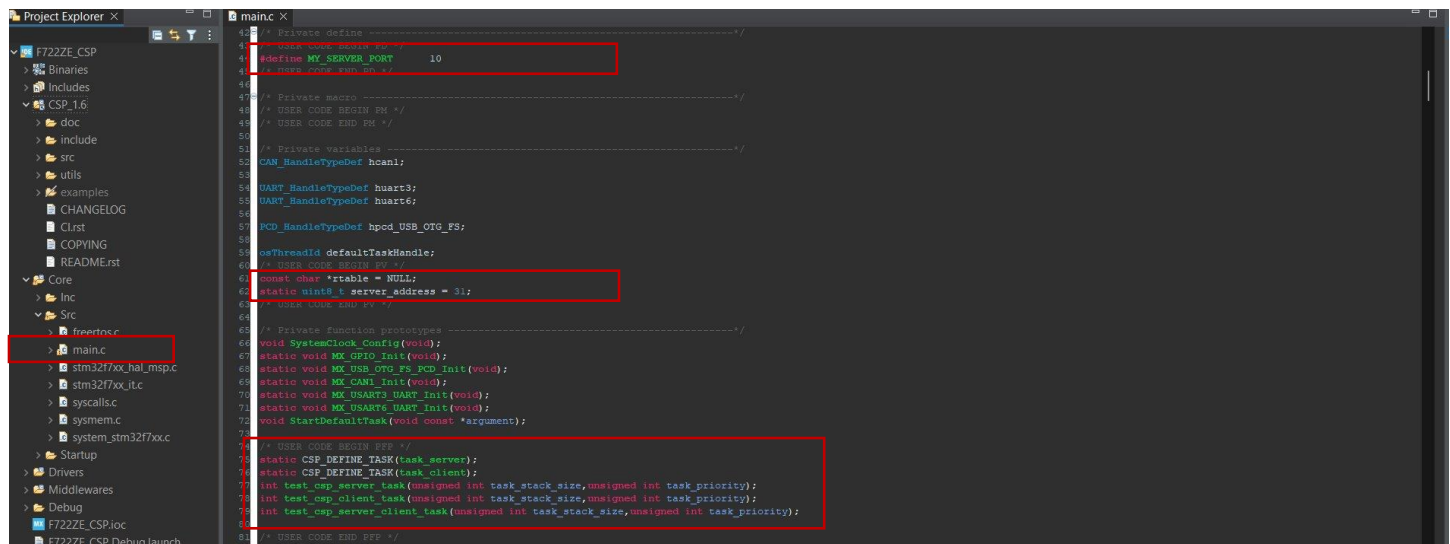


Figure 16 Define variables and create functions

For debugging print, display data via UART

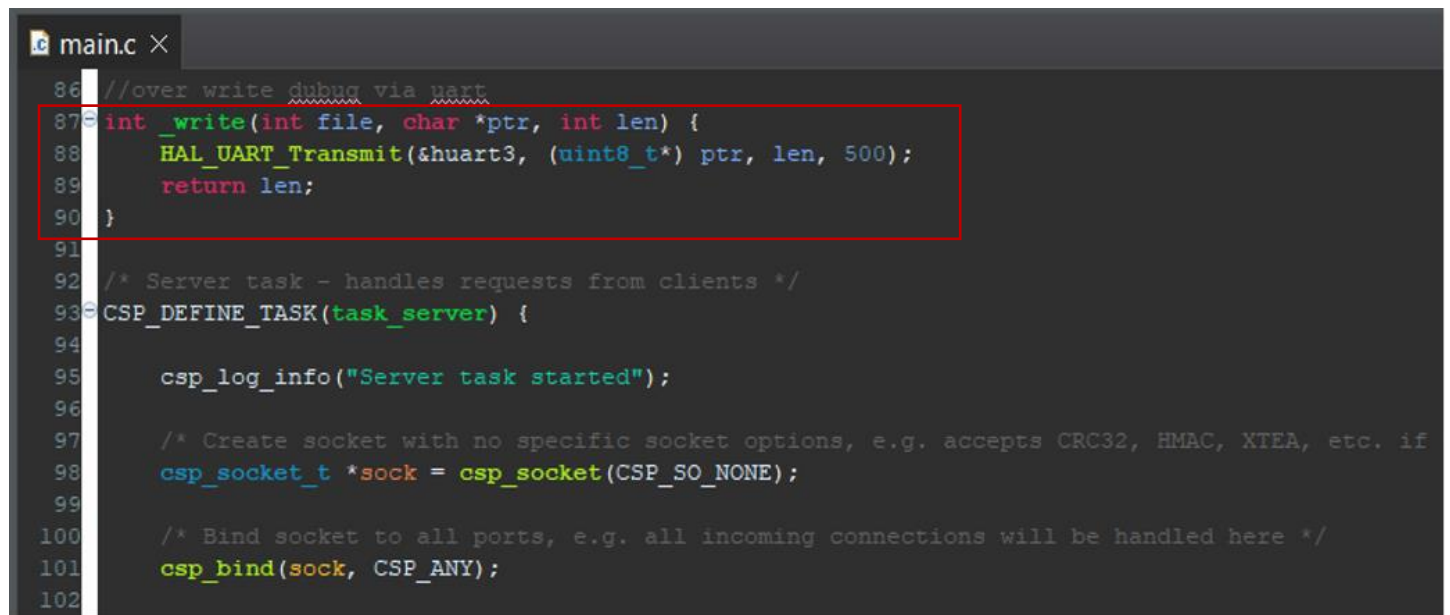


Figure 17 debugging print

Create a function for testing server and client

Example code is available in the examples file **CSP_1.6/Examples/csp_server_client.c**

Create task server

```
87 CSP_DEFINE_TASK(task_server) {
88
89     csp_log_info("Server task started");
90
91     /* Create socket with no specific socket options, e.g. accepts CRC32, HMAC, XTEA, etc. if enabled during compilation */
92     csp_socket_t *sock = csp_socket(CSP_SO_NONE);
93
94     /* Bind socket to all ports, e.g. all incoming connections will be handled here */
95     csp_bind(sock, CSP_ANY);
96
97     /* Create a backlog of 10 connections, i.e. up to 10 new connections can be queued */
98     csp_listen(sock, 10);
99
100    /* Wait for connections and then process packets on the connection */
101    while (1) {
102
103        /* Wait for a new connection, 10000 ms timeout */
104        csp_conn_t *conn;
105        if ((conn = csp_accept(sock, 10000)) == NULL) {
106            /* timeout */
107            continue;
108        }
109
110        /* Read packets on connection, 10000 is 100 ms */
111        csp_packet_t *packet;
112        while ((packet = csp_read(conn, 50)) != NULL) {
113            switch (csp_conn_dport(conn)) {
114                case MY_SERVER_PORT:
115                    /* Process packet here */
116                    csp_log_info("Packet received on MY_SERVER_PORT: %s", (char *) packet->data);
117                    csp_buffer_free(packet);
118                    break;
119
120                default:
121                    /* Call the default CSP service handler, handle pings, buffer use, etc. */
122                    csp_service_handler(conn, packet);
123                    break;
124            }
125        }
126
127        /* Close current connection */
128        csp_close(conn);
129    }
130
131    return CSP_TASK_RETURN;
132
133 }
134
135 /* End of server task */
```

Figure 18 task server

Create task client

```
main.c X
144 CSP_DEFINE_TASK(task_client) {
145
146     csp_log_info("Client task started");
147
148     unsigned int count = 0;
149
150     while (!) {
151
152         csp_sleep_ms(1000);
153
154         /* Send ping to server, timeout 1000 ms, ping size 100 bytes */
155         int result = csp_ping(server_address, 1000, 100, CSP_O_NONE);
156         csp_log_info("Ping address: %u, result %d [ms]", server_address, result);
157
158         /* Send reboot request to server, the server has no actual implementation of csp_sys_reboot() and fails to reboot */
159         csp_reboot(server_address);
160         csp_log_info("Reboot system request sent to address: %u", server_address);
161
162         /* Send data packet (string) to server */
163
164         /* 1. Connect to host on 'server_address', port MY_SERVER_PORT with regular UDP-like protocol and 1000 ms timeout */
165         csp_conn_t * conn = csp_connect(CSP_FRIQ_NORM, server_address, MY_SERVER_PORT, 1000, CSP_O_NONE);
166         if (conn == NULL) {
167             /* Connect failed */
168             csp_log_error("Connection failed");
169             return CSP_TASK_RETURN;
170         }
171
172         /* 2. Get packet buffer for message/data */
173         csp_packet_t * packet = csp_buffer_get(100);
174         if (packet == NULL) {
175             /* Could not get buffer element */
176             csp_log_error("Failed to get CSP buffer");
177             return CSP_TASK_RETURN;
178         }
179
180         /* 3. Copy data to packet */
181         snprintf((char *) packet->data, csp_buffer_data_size(), "Hello World (%u)", ++count);
182
183         /* 4. Set packet length */
184         packet->length = (strlen((char *) packet->data) + 1); /* include the 0 termination */
185
186         /* 5. Send packet */
187         if (!csp_send(conn, packet, 1000)) {
188             /* Send failed */
189             csp_log_error("Send failed");
190             csp_buffer_free(packet);
191         }
192
193         /* 6. Close connection */
194         csp_close(conn);
195     }
```

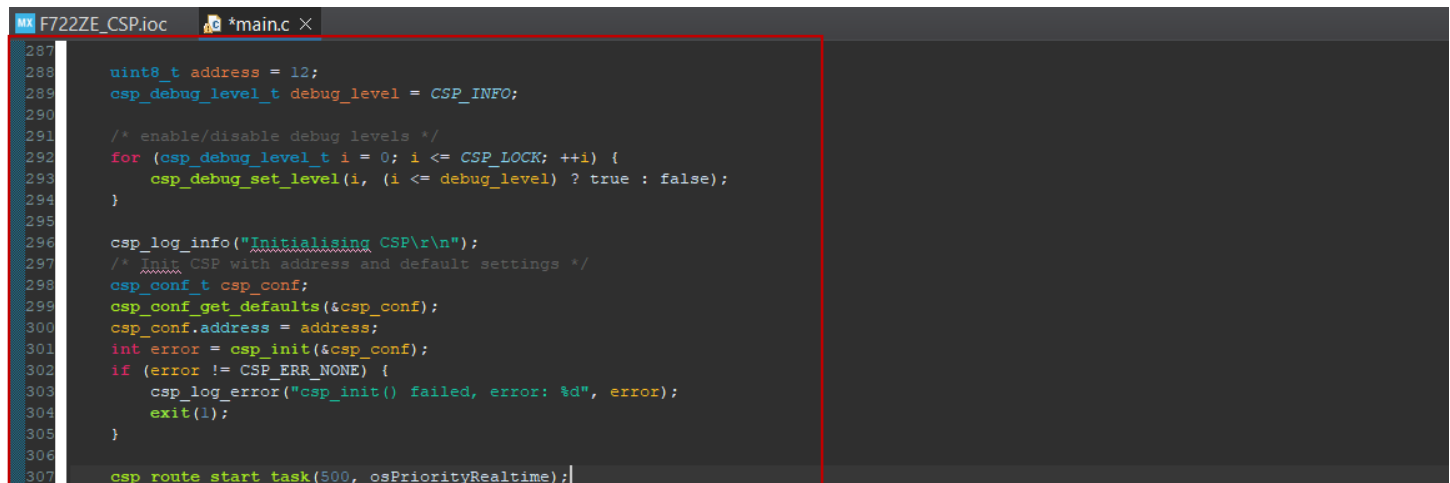
Figure 19 task client

Create a function for creating tasks

```
main.c X
201 int test_csp_server_task(unsigned int task_stack_size,
202     unsigned int task_priority) {
203     int ret = csp_thread_create(task_server, "SERVER", task_stack_size, NULL,
204         task_priority, NULL);
205     if (ret != 0) {
206         csp_log_error("Failed to start server task, error: %d", ret);
207         return ret;
208     }
209     return CSP_ERR_NONE;
210 }
211
212 int test_csp_client_task(unsigned int task_stack_size,
213     unsigned int task_priority) {
214     int ret = csp_thread_create(task_client, "CLIENT", task_stack_size, NULL,
215         task_priority, NULL);
216     if (ret != 0) {
217         csp_log_error("Failed to start client task, error: %d", ret);
218         return ret;
219     }
220     return CSP_ERR_NONE;
221 }
222
223 int test_csp_server_client_task(unsigned int task_stack_size,
224     unsigned int task_priority) {
225     int ret1 = csp_thread_create(task_server, "SERVER", task_stack_size, NULL,
226         task_priority, NULL);
227     if (ret1 != 0) {
228         csp_log_error("Failed to start server task, error: %d", ret1);
229         return ret1;
230     }
231     int ret2 = csp_thread_create(task_client, "CLIENT", task_stack_size, NULL,
232         task_priority, NULL);
233     if (ret2 != 0) {
234         csp_log_error("Failed to start client task, error: %d", ret2);
235         return ret2;
236     }
237     return CSP_ERR_NONE;
238 }
```

Figure 20 function for creating tasks

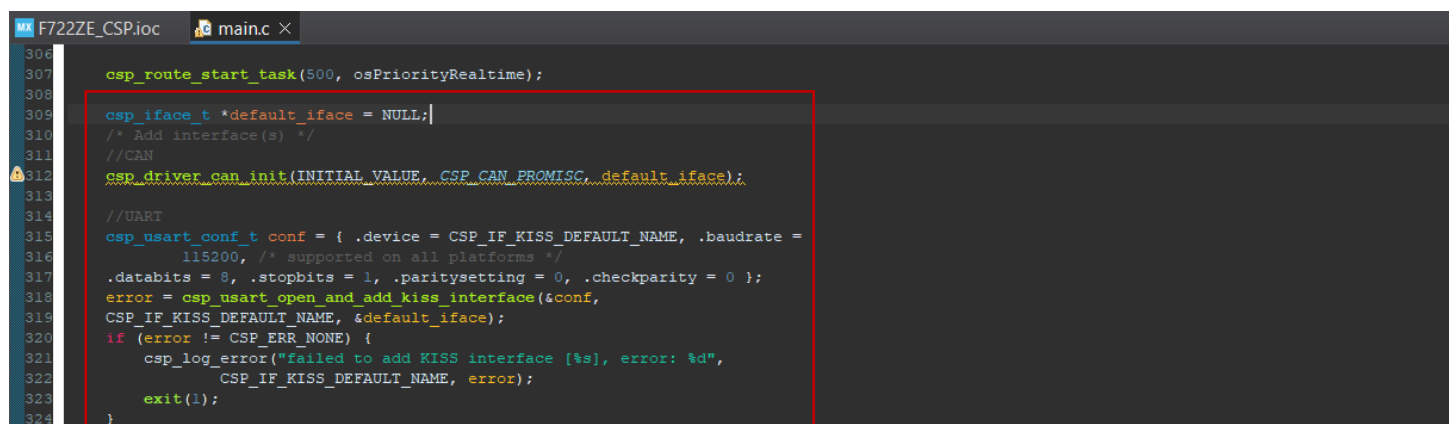
3.5 Demonstrate initialization and adding interfaces, enabling task for testing in the main.c file.



```
F722ZE_CSP.ioc  main.c ×
287
288     uint8_t address = 12;
289     csp_debug_level_t debug_level = CSP_INFO;
290
291     /* enable/disable debug levels */
292     for (csp_debug_level_t i = 0; i <= CSP_LOCK; ++i) {
293         csp_debug_set_level(i, (i <= debug_level) ? true : false);
294     }
295
296     csp_log_info("Initialising CSP\r\n");
297     /* init CSP with address and default settings */
298     csp_conf_t csp_conf;
299     csp_conf_get_defaults(&csp_conf);
300     csp_conf.address = address;
301     int error = csp_init(&csp_conf);
302     if (error != CSP_ERR_NONE) {
303         csp_log_error("csp_init() failed, error: %d", error);
304         exit(1);
305     }
306
307     csp_route_start_task(500, osPriorityRealtime);
```

Figure 21 initialization and adding interfaces

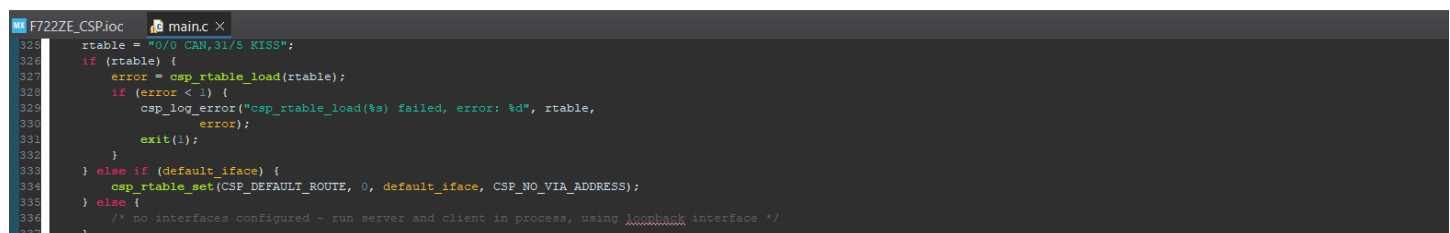
3.6 add interfaces



```
F722ZE_CSP.ioc  main.c ×
306
307     csp_route_start_task(500, osPriorityRealtime);
308
309     csp_iface_t *default_iface = NULL;
310     /* Add interface(s) */
311     //CAN
312     csp_driver_can_init(INITIAL_VALUE, CSP_CAN_PROMISC, default_iface);
313
314     //UART
315     csp_uart_conf_t conf = { .device = CSP_IF_KISS_DEFAULT_NAME, .baudrate =
316         115200, /* supported on all platforms */
317         .databits = 8, .stopbits = 1, .paritysetting = 0, .checkparity = 0 };
318     error = csp_uart_open_and_add_kiss_interface(&conf,
319         CSP_IF_KISS_DEFAULT_NAME, &default_iface);
320     if (error != CSP_ERR_NONE) {
321         csp_log_error("failed to add KISS interface [%s], error: %d",
322             CSP_IF_KISS_DEFAULT_NAME, error);
323         exit(1);
324     }
```

Figure 22 add interfaces

3.7 Routing Configuration



```
F722ZE_CSP.ioc  main.c ×
325     rtable = "0/0 CAN,31/5 KISS";
326     if (rtable) {
327         error = csp_rtable_load(rtable);
328         if (error < 1) {
329             csp_log_error("csp_rtable_load(%s) failed, error: %d", rtable,
330                 error);
331             exit(1);
332         }
333     } else if (default_iface) {
334         csp_rtable_set(CSP_DEFAULT_ROUTE, 0, default_iface, CSP_NO_VIA_ADDRESS);
335     } else {
336         /* no interfaces configured - run server and client in process, using localhost interface */
337     }
```

Figure 23 Routing Configuration

Enabling CSP Testing Task

```
339     printf("Connection table\r\n");
340     csp_conn_print_table();
341
342     printf("Interfaces\r\n");
343     csp_route_print_interfaces();
344
345     printf("Route table\r\n");
346     csp_route_print_table();
347
348     //test_csp_server_Client(500, osPriorityAboveNormal);
349     //test_csp_server_task(500, osPriorityAboveNormal);
350     //test_csp_client_task(500, osPriorityAboveNormal);
351
352     /* USER CODE END RTOS_THREADS */
```

Figure 24 Enabling CSP Testing Task

4. Uploading to Nucleo and debugging.

Compiling the Code: First, you need to compile your code to generate the binary file that will be uploaded to the Nucleo board. This step involves building your project within the development environment you're using, such as STM32CubeIDE or Keil μ Vision.

Connecting the Nucleo Board: Connect your Nucleo board to your computer using a USB cable. Ensure that the board is recognized by your operating system and that the necessary drivers are installed.

Uploading the Firmware: Use a programming tool or debugger, such as ST-Link or J-Link, to upload the compiled firmware to the Nucleo board. This is typically done using the programming/debugging interface provided on the Nucleo board.

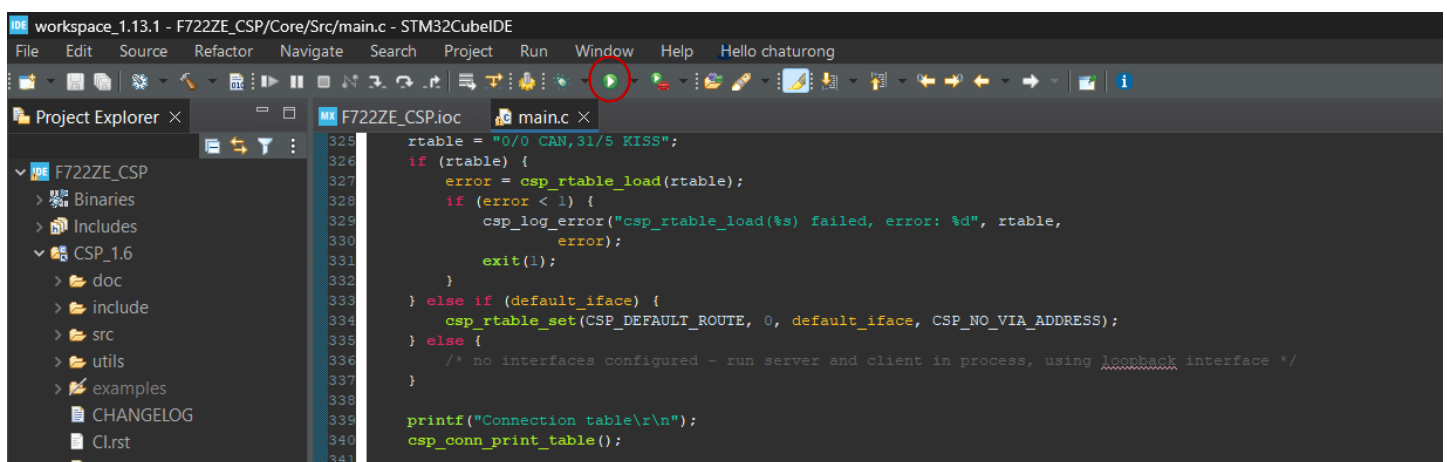


Figure 25 Uploading to Nucleo